Chapter 5 — Recursion and Recurrence Relations

Course notes summary · recursion, program stack, tail recursion, space complexity, recurrence relations, iterative substitution, Master Theorem

How to read these notes: ★ NOT TESTED = purple dashed box, safe to skim (included for completeness). Blue box = definition/key rule. Green box = worked example. Yellow box = remark / common pitfall.
Chapter status:

- You do **NOT** need to memorize any **summation formulas** — the math is provided for completeness (formulas are shown in purple boxes).
- **End of 5.7 (deriving the Master Theorem with recursion trees)** — not required, but it makes the theorem much easier to understand. Memorizing the three conditions is fine.
- **5.7.3 (Extended Master Theorem for polylogarithmic f(n))** — not required for exams.

5.1 Recursion

Definitions

- A function is **recursive** if it calls itself in its own implementation.
- **Linear recursive:** each invocation makes **at most one** recursive call.
- **Tree recursive:** each invocation can make **more than one** recursive call.
- **Base case:** an input size small enough that the problem is solved trivially, with no further recursive calls. It lets the solution build back up to the original input size.
- **Recursive leap of faith:** assume your recursive function already works on the smaller subproblem.

- Recursion is useful when the solution to a **smaller subproblem of the same structure** can be used to solve a larger one.
- Two-step recipe:
  1. Assume the function already works; make a recursive call on a smaller subproblem to get its solution (leap of faith).
  2. Use that smaller solution to solve the original problem.
- Soundness rests on: calling yourself with smaller and smaller inputs will **eventually reach the base case**.

Intuition — people waiting in line

You want to know your position in a long line but cannot leave it.

- You assume the person in front knows their position (= **recursive leap of faith**) and ask them (= **recursive call with smaller input**).
- They don't know either, so they ask the person in front of them, and so on.
- The **first person in line** trivially knows they are first (= **base case**).
- Answers flow back down the line, each person adding one, until you get yours.

```
int32_t get_line_position(int32_t person) {
  // if first person, return one (base case)
  if (person == 1) {
    return 1;
  } // if
  // otherwise, return one plus the position of the person before you
  return 1 + get_line_position(person - 1);
} // get_line_position()
```

Factorial

Mathematical definition in terms of itself (by definition 0! = 1):

$$n! = 1 \text{ if } n = 0 \quad | \quad n \times (n-1)! \text{ if } n > 0$$

```
int32_t factorial(int32_t n) {
  if (n == 0) {
    return 1;             // base case
  } // if
  return n * factorial(n - 1);
} // factorial()
```

Trace for n = 10: factorial(10) returns `10 * factorial(9)`, which triggers factorial(9) → factorial(8) → … → factorial(0), which hits the base case and returns 1 with no further calls. The values then multiply back up.

Key observation: after calling factorial(9), we still need `n` to multiply by. Because `n` must be referenced **after** the recursive call, it has to be saved somewhere in memory → the **program stack**.

5.2 The Program Stack

- If function A calls function B, we must remember A's state (local variables, which line we were on) so we can resume A when B finishes.
- That information is stored in **stack frames** on the **program stack**.
- The **stack** holds local variables and bookkeeping data (vs. the **heap**, which is for dynamic memory).
- Stack access is **LIFO** (last-in first-out): the most recently allocated block is the first one removed.

Mechanics of a call and a return

On a function call:

1. The caller's local variables get stored on the program stack.
2. The function arguments are pushed onto the stack.
3. Control transfers from caller to callee.
4. The new function pops the arguments off the stack and starts running.

On a return:

1. The return value is pushed onto the stack.
2. The caller resumes execution.
3. The caller pops the return value off the stack and restores its local variables.

A frame's block of memory is called an **activation record**: local variables, temporary objects, the return address, and other bookkeeping.

Full trace of `factorial(3)` called from main()

#	Active frame	What happens
1	main()	Frame allocated for main(). Line 9 creates uninitialized `res`; to assign it, calls factorial(3). main()'s state saved; argument 3 pushed.
2	factorial(3)	Frame allocated; pops 3 into `n`. n ≠ 0 so the `if` is skipped; line 5 calls factorial(2). State saved; argument 2 pushed.
3	factorial(2)	Frame allocated; pops 2 into `n` (local to **this** call). Calls factorial(1); argument 1 pushed.
4	factorial(1)	Frame allocated; pops 1 into `n`. Calls factorial(0); argument 0 pushed.
5	factorial(0)	Frame allocated; n = 0 → **base case** returns 1. Return value pushed; frame **deallocated**.
6	factorial(1)	Most recent frame resumes on line 5; locals restored; 1 * 1 = 1 returned; frame deallocated.
7	factorial(2)	Resumes; 1 * 2 = 2 returned; frame deallocated.
8	factorial(3)	Resumes; 2 * 3 = 6 returned; frame deallocated.
9	main()	Resumes on line 9; `res` = 6; prints "3! = 6"; returns 0 (the program's **exit status**, 0 = success). main()'s frame deallocated; program completes.

Stack at its deepest (step 5), grows upward:

```
+----------------+
| factorial(0)  n=0 |   <- top (most recent)
| factorial(1)  n=1 |
| factorial(2)  n=2 |
| factorial(3)  n=3 |
| main()      res=? |   <- bottom
+----------------+
```

Stack overflow

The program stack is **limited**. Recursing too deeply exhausts the available stack space and causes a **stack overflow** error. One strategy for avoiding it: **tail recursion**.

5.3 Tail Recursion

Definition

A **tail recursive** function is a **linear recursive** function where **the recursive call is the final instruction of the function**.

- Since nothing happens after the call returns, the function's local variables will never be needed again → no need to remember them.
- → The compiler can **optimize memory and reuse the same stack frame** for every recursive call.

Not tail recursive

```
int32_t factorial(int32_t n) {
  if (n == 0) { return 1; } // if
  return n * factorial(n - 1);   // must multiply by n AFTER the call returns
} // factorial()
```

- The result must be multiplied by `n` after the call completes, so `n` must be remembered → a separate frame per call.

Tail recursive (with an accumulator argument)

```
int32_t factorial(int32_t n, int32_t res = 1) {
  if (n == 0) { return res; } // if
  return factorial(n - 1, n * res);   // multiplication done IN the argument
} // factorial()
```

- An **accumulator argument** lets you finish the multiplication **in the argument of the recursive call**, before the call returns.
- Nothing is left to do after the call returns → tail recursive → the same stack frame is reused.

Remark — default arguments

`int32_t res = 1` in the signature is a **default argument**: calling `factorial(3)` without a second argument automatically sets `res` to 1.

Iteration vs. recursion

- Both achieve repetition. Any **iterative** function can be converted to a recursive one, because **iteration is simply a special case of tail recursion**.
- Any **recursive** function can be converted to an iterative one by **simulating the program stack** yourself (e.g. keeping a `std::stack<>`).
- You will not be required to know how to convert between the two.

5.4 Stack Frames and Space Complexity

Core rule

- **Non-tail recursive:** auxiliary space is determined by the **recursion depth** = the number of **return statements** that must execute until the base case is reached. Depth of n → **Θ(n)** auxiliary space.
- **Tail recursive:** the frame is reused, so the number of frames does **not** grow with depth → **Θ(1)** auxiliary space for stack frames.
- Count **return statements until the base case**, **not** the total number of recursive calls made — frames for different calls may not exist in memory at the same time.
- Also count auxiliary space used **outside** the frames: Θ(n) frames + a Θ(n^2) container = Θ(n + n^2) = **Θ(n^2)**.

Example 5.1 — two recursive calls, but not 2^n space

```
int32_t foo(int32_t n) {
  if (n <= 1) { return 1; } // if
  return foo(n - 1) + foo(n - 1);
} // foo()
```

Tempting (wrong) answer: Θ(2^n) calls are made, so Θ(2^n) space. But the frames **do not all coexist**.
Trace of `foo(4)` down its leftmost path:

```
foo(4)            foo(4)            foo(4)
                  foo(3)            foo(3)
                                    foo(2)
                                    foo(1)   <- base case, 4 frames max
```

foo(1) returns and its frame is **deallocated** before foo(2) makes its second call to foo(1) — those two frames never coexist.
So even though 2^4 = 16 calls happen, at most **4** frames exist at once. n is decremented per call → n return statements before the base case → **Θ(n)** auxiliary space.

Example 5.2 — tail recursive → Θ(1)

```
void print_to_n(int32_t n) {
  if (n < 1) { return; } // if
  std::cout << n << " ";
  print_to_n(n - 1);       // last thing done
} // print_to_n()
```

Recursion depth is n (decrement from n to 1; the return of a void function is implicit). But the recursive call is the **very last** instruction → tail recursive → frame reused. No other memory depends on n. **Auxiliary space = Θ(1)**.

Example 5.3 — halving input, not tail recursive

```
int32_t bar(int32_t n) {
  if (n < 1) { return; } // if
  return n + bar(n / 2);   // must add n AFTER the call
} // bar()
```

Input halved each call → recursion depth = **log(n)**. The call is **not** last (the result must still be added to n) → frames cannot be reused. **Auxiliary space = Θ(log(n))**.

Summary

Analyze **both stack and heap** memory. A non-tail recursive function that never explicitly allocates memory can still allocate more and more stack frames as input size grows, as long as recursion depth grows with input size.

5.5 Recurrence Relations

Definition

A **recurrence relation** is an equation that defines the runtime T(n) of a problem **in terms of the runtime of a recursive call on a smaller input**.

Example — the non-tail recursive factorial (return 1 if n == 0, else recurse on n−1):

$$T(n) = \Theta(1) \text{ if } n = 0 \quad | \quad T(n-1) + \Theta(1) \text{ if } n > 0$$

- T(n) = runtime of `factorial(n)`; T(n−1) = runtime of the recursive call; the extra Θ(1) = constant work done outside the recursive call.
- The exact constant does not matter, since the relation is only used to derive a complexity class.

How to build a recurrence relation from code

Same procedure as chapter 4:

- Operations that **follow one another** → complexities are **added**.
- Operations **nested inside loops** → complexities are **multiplied** by the loop count.
- When you reach a **recursive call** → add a term T(<input size of that call>).
- A recursive call made inside a loop that runs n times contributes **n T(…)**.

Example 5.4 — write the recurrence for foo()

Assume `bar()` runs in Θ(log(n)) time and n ≥ 1.

```
1 void foo(int32_t n) {
2   if (n == 1) {
3     return;
4   } // if
5   foo(n - 1);
6   int k = n * n;
7   for (int i = 0; i < k; ++i) {
8     for (int j = 0; j < n; ++j) {
9       bar(n);
10    } }
12  for (int i = 0; i < n; ++i) {
13    foo(n / 2);
14  } // for i
15  bar(k);
16 } // foo()
```

- **L2–3:** constant Θ(1), but only runs when n = 1 → this is the base case.
- **L5:** recursive call on n−1 → term **T(n−1)**.
- **L6:** constant Θ(1).
- **L7–9:** bar() is Θ(log n); inner loop runs n times; outer loop runs $k = n^2$ times → Θ(log(n) × n × n^2) = **$\Theta(n^3 \log(n))$**.
- **L12–13:** a T(n/2) call made n times → **n T(n/2)**.
- **L15:** bar() with input n^2 → Θ(log(n^2)) = **2 log(n)**.

$$T(n) = \Theta(1), \text{ if } n = 1$$
$$T(n) = T(n-1) + n^3\log(n) + n\,T(n/2) + 2\log(n) + \Theta(1), \text{ if } n > 1$$

5.6 The Iterative Substitution Method

- Two methods in this chapter for turning a recurrence into a complexity class: **iterative substitution** (this section) and the **Master Theorem** (5.7).
- **Motivation:** you cannot read the complexity off 2T(n−1) + Θ(1) directly. But you **do** know the base case. If you can rewrite the relation so that only the base case appears, you can substitute in its constant value and eliminate all recursive terms. That result is a **closed form solution**.

The three steps

1. Write out the recursive terms (T(n−1), T(n−2), ...) as their own recurrence relations and **substitute** them into the original T(n) formula at each step.
2. Look for a **pattern that describes T(n) at the k^{th} step** (for arbitrary k) and express it with a summation formula.
3. **Solve for k** so that the base case is the only recursive term left on the right-hand side. Replace the base case with its value (e.g. T(1) = 1) to get the closed form.

We substitute Θ(1) with the constant 1 throughout — it simplifies the math without changing the result. (T(1) runs in constant time, so any constant works.)

Common mistake

When writing the recurrence for T(n−1), you must replace **ALL** instances of n with (n−1) — including terms **outside** the recursive call, not just inside it.
Correct: if T(n) = T(n−1) + log(n), then T(n−1) = T(n−2) + log(n−1).
Wrong: T(n−1) = T(n−2) + log(n).

Example 5.5 — T(n) = 2T(n−1) + 1 → Θ(2^n)

```
int32_t foo(int32_t n) {
  if (n == 1) { return 1; } // if
  return foo(n - 1) + foo(n - 1) + 1;
} // foo()
```

$$T(n) = 1 \text{ if } n = 1 \;\mid\; 2T(n-1) + 1 \text{ if } n > 1$$

Step 1 — substitute:

Step	Recurrence equation
1	T(n) = 2 × T(n−1) + 1
2	T(n−1) = 2T(n−2)+1 ⇒ T(n) = 2 × 2 × T(n−2) + 2 + 1
3	T(n−2) = 2T(n−3)+1 ⇒ T(n) = 2^3 × T(n−3) + 4 + 2 + 1
4	T(n−3) = 2T(n−4)+1 ⇒ T(n) = 2^4 × T(n−4) + 8 + 4 + 2 + 1

Step 2 — generalize to the k^{th} step:

$$T(n) = 2^k T(n-k) + \Sigma_{i=0}^{k-1}\, 2^i$$

Step 3 — solve for k: we want T(n−k) = T(1), so n−k = 1 → k = n−1. Plug in T(1) = 1:

$$T(n) = 2^{n-1}(1) + \Sigma_{i=0}^{n-2}\, 2^i$$

Using the geometric identity with a = 1, r = 2: $\Sigma_{i=0}^{n-2} 2^i = 2^{n-1} - 1$, so

$$T(n) = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1 \;\rightarrow\; \mathbf{\Theta(2^n)}$$

★ Summation identities used in this chapter — NOT required to memorize

- $\Sigma_{i=m}^{n}\, ar^i = a(r^m - r^{n+1}) / (1 - r)$ (geometric, used in Ex. 5.5)
- $S_n = a_1(1 - r^n)/(1 - r)$ (finite geometric series)
- $\Sigma_{i=1}^{n}\, n!/(n-i)! = \lfloor n!e \rfloor - 1$, $e \approx 2.71828$ (used in Ex. 5.7)
- $\Sigma_{i=0}^{n}\, i2^i = 2^{n+1}n - 2^{n+1} + 2$ (used in Ex. 5.23 algorithm 3)
- Arithmetic sum: $n((a_1 + a_n)/2)$ (used in Ex. 5.21)

These will be given to you if needed. What matters is the **process**, not the formulas.

Example 5.6 — T(n) = T(n−1) + log(n) → Θ(n log(n))

```
void foo(int32_t n) {
  if (n == 0) { return; } // if
  for (int32_t i = 1; i < n; i *= 2) {
    std::cout << "281" << std::endl;
  } // for i
  foo(n - 1);
} // foo()
```

Loop doubles i → log(n) iterations; one recursive call on n−1:

$$T(n) = 1 \text{ if } n = 0 \;\mid\; T(n-1) + \log(n) \text{ if } n > 0$$

Technically it is T(n−1) + log(n) + 1, but the constant is dominated by log(n) and can be dropped. **Steps 1–2:**

- Step 1: T(n) = T(n−1) + log(n)
- Step 2: T(n) = T(n−2) + log(n−1) + log(n)
- Step 3: T(n) = T(n−3) + log(n−2) + log(n−1) + log(n)
- Step 4: T(n) = T(n−4) + log(n−3) + log(n−2) + log(n−1) + log(n)

$$k^{\text{th}} \text{ step: } T(n) = T(n-k) + \Sigma_{i=0}^{k-1} \log(n-i)$$

Step 3: base case is T(0), so n−k = 0 → k = n; plug in T(0) = 1:

$$T(n) = 1 + \Sigma_{i=0}^{n-1} \log(n-i)$$

Sum of logs = log of the product:

$$T(n) = 1 + \log(n \times (n-1) \times \ldots \times 2 \times 1) = 1 + \log(n!) = \Theta(\log(n!))$$

From chapter 4, log(n!) = Θ(n log(n)) → **Θ(n log(n))** (which matches the intuition: log(n) work, n recursive calls).

Example 5.7 — T(n) = nT(n−1) + n → Θ(n!)

```
int64_t foo(int64_t n) {
  int64_t s = 1;
  if (n == 0) { return s; } // if
  for (int64_t i = 1; i <= n; ++i) {
    s += foo(n - 1);
  } // for i
  return s;
} // foo()
```

The loop runs a recursive call n times plus constant work:

$$T(n) = 1 \text{ if } n = 0 \;\mid\; nT(n-1) + n \text{ if } n > 0$$

- Step 1: T(n) = nT(n−1) + n
- Step 2: T(n) = n(n−1)T(n−2) + n(n−1) + n
- Step 3: T(n) = n(n−1)(n−2)T(n−3) + n(n−1)(n−2) + n(n−1) + n

$$k^{\text{th}} \text{ step: } T(n) = [n!/(n-k)!]\, T(n-k) + \Sigma_{i=1}^{k}\, n!/(n-i)!$$

Base case T(0) → k = n; T(0) = 1; apply the given identity:

$$T(n) = n! + \Sigma_{i=1}^{n}\, n!/(n-i)! = \lfloor n!(e+1) \rfloor - 1 \approx 3.71828\, n! - 1 \rightarrow \mathbf{\Theta(n!)}$$

Example 5.8 — T(n) = 2T(n/2) + n → Θ(n log(n))

```
void foo(int32_t n) {
  if (n == 1) { return; } // if
  foo(n / 2);
  for (int32_t i = 0; i < n; ++i) {
    std::cout << "281" << std::endl;
  } // for i
  foo(n / 2);
} // foo()
```

$$T(n) = 1 \text{ if } n = 1 \;\mid\; 2T(n/2) + n \text{ if } n > 1$$

- Step 1: T(n) = 2T(n/2) + n
- Step 2: T(n/2) = 2T(n/4) + n/2 ⇒ T(n) = 4T(n/4) + 2n
- Step 3: T(n/4) = 2T(n/8) + n/4 ⇒ T(n) = 8T(n/8) + 3n

$$k^{\text{th}} \text{ step: } T(n) = 2^k T(n/2^k) + kn$$

Base case T(1) → $n/2^k = 1 \rightarrow 2^k = n \rightarrow k = \log_2(n)$; plug in T(1) = 1:

$$T(n) = 2^{\log_2(n)} T(1) + n \log_2(n) = n + n \log_2(n) \rightarrow \mathbf{\Theta(n \log(n))}$$

Remark — rounding
By convention T is only defined on integer arguments, so T(n/2) really means T(⌊n/2⌋) when n is odd. For big-O bounds this rounding makes no difference, so the **floor is usually implicitly assumed** whenever division appears in a recursive term.

5.7 The Master Theorem

5.7.1 Applying the Master Theorem

The Master Theorem
If the recurrence relation has the form

$$T(n) = a\,T(n/b) + f(n)$$

where **$a \geq 1$**, **$b > 1$**, and f(n) is a positive function that is **$\Theta(n^c)$**, then:

Condition	Complexity of T(n)
$a > b^c$	$\Theta(n^{\log_b(a)})$
$a = b^c$	$\Theta(n^c \log(n))$
$a < b^c$	$\Theta(n^c)$

Meaning of the variables: **a** = number of recursive calls made; **b** = the factor the input is divided by per call; **c** = the highest power of the polynomial f(n).
Conditions that must all hold:

- There must exist a **base case solvable in constant time**.
- a and b **cannot depend on n**.
- The recursive argument must be **divided** by some b > 1 (not, e.g., n−1).
- f(n) must be asymptotically positive and of the form $\Theta(n^c)$ (polynomial).
- Only **one** kind of recursive call (a single b value).

Steps
1. Determine a, b, c.
2. Verify the Master Theorem can be used (the conditions above).
3. Compare **a** with **b^c** to pick the case.

Ex.	Recurrence (n > 1; base case T(1) = 1)	a, b, c	Compare	Result
5.9	$3T(n/2) + 5n + 13$	a=3, b=2, c=1 ($f = \Theta(n^1)$)	$3 > 2^1=2$	$\Theta(n^{\log_2(3)}) \approx \Theta(n^{1.585})$
5.10	$4T(n/2) + 5n + 13n^2$	a=4, b=2, c=2	$4 = 2^2=4$	$\Theta(n^2 \log(n))$
5.11	$6T(n/2) + 5n + 2^n$	$f(n) = \Theta(2^n)$	—	**Cannot use** — f(n) is not polynomial
5.12	$6T(n/2) + 3T(n/5) + 4n + 3$	two different b values	—	**Cannot use** — two different recursive calls
5.13	$T(n/4) + n\sqrt{n}$	a=1, b=4, c=3/2	$1 < 4^{3/2}=8$	$\Theta(n^{3/2})$
5.14	$5T(n/8) + 15$	a=5, b=8, c=0 ($15 = 15n^0$)	$5 > 8^0=1$	$\Theta(n^{\log_8(5)})$
5.15	$7T(10n/11) + 9n$	a=7, b=11/10=1.1, c=1	7 > 1.1	$\Theta(n^{\log_{1.1}(7)}) \approx \Theta(n^{20.417})$
5.16	$3T(n/2) + \Theta(n)$ — **Karatsuba** multiplication	a=3, b=2, c=1	3 > 2	$\Theta(n^{\log_2(3)}) \approx \Theta(n^{1.585})$
5.17	$7T(n/2) + \Theta(n^2)$ — **Strassen** matrix multiply	a=7, b=2, c=2	7 > 4	$\Theta(n^{\log_2(7)})$

Ex. 5.15 note: 10n/11 is rewritten as n/(11/10), so b = 1.1. **Ex. 5.16:** Karatsuba multiplies two n-digit numbers faster than grade-school multiplication. **Ex. 5.17:** Strassen multiplies two n×n matrices faster than standard matrix multiplication.

5.7.2 Deriving the Master Theorem ★ NOT REQUIRED

★ Intuition only. Memorizing the three conditions above is perfectly fine for this class — but this makes them easier to remember.

Recursion tree: a tree illustrating the work done by a recursive algorithm. Each **branch** = a recursive call; each **node** = the additional work done at that call. The **sum of all nodes** = total work.

```
      10          <- 10 units of work here, then two calls
     /    \
   20      30     total work = 10 + 20 + 30 = 60
```

Building one: (1) each node is assigned the work done **outside** the recursive call(s); (2) each recursive call creates a branch to the next level. Repeat until the base case.
For T(n) = 2T(n/2) + n:

```
        n
      /   \
    n/2   n/2
   /  \   /  \
 n/4 n/4 n/4 n/4      ... down to the base case
```

Case a > b^c: "bottom-heavy" / "leaf-heavy" — e.g. T(n) = 4T(n/2) + n.

Level	Work at that level
T(n)	n
T(n/2)	4(n/2) = 2n
T(n/4)	16(n/4) = 4n

Work **increases** at each level (n < 2n < 4n < ...) even though f(n) shrinks, because a is so large that the sheer number of recursive calls dominates. Asymptotically **all** the work happens at the lowest level (the leaves).

- The tree has $\log_b(n)$ levels (input divided by b until the base case).
- Number of nodes at the last level = $a^{\log_b(n)}$. Using $a = n^{\log_n(a)}$ and change of base:

$$a^{\log_b(n)} = n^{\log_n(a)\,\log_b(n)} = n^{\log_b(a)}$$

  (since $\log_n(a)\log_b(n) = [\log(a)/\log(n)] \times [\log(n)/\log(b)] = \log(a)/\log(b) = \log_b(a)$)
- Each leaf is a base case doing constant work → total = **$\Theta(n^{\log_b(a)})$**, matching the theorem.

Case a < b^c: "top-heavy" / "root-heavy" — e.g. T(n) = 2T(n/2) + n^2.

Level	Work at that level
T(n)	n^2
T(n/2)	$2(n^2/4) = n^2/2$
T(n/4)	$4(n^2/16) = n^2/4$

Work **decreases** at each level. The splitting/recombining work f(n) dwarfs the recursive subproblem work, so asymptotically all the work is at the **root** → **$\Theta(f(n)) = \Theta(n^c)$**.

Case a = b^c: balanced — e.g. T(n) = 4T(n/2) + n^2.

Level	Work at that level
T(n)	n^2
T(n/2)	$4(n^2/4) = n^2$
T(n/4)	$16(n^2/16) = n^2$

Every level does $\Theta(n^c)$ work, and there are $\log_2(n)$ levels → **$\Theta(n^c \log(n))$**.

One-line summary: a > b^c → leaves dominate; a < b^c → root dominates; a = b^c → every level costs the same and there are log(n) of them.

5.7.3 Extended Master Theorem for Polylogarithmic Functions ★ NOT TESTED

★ You do NOT need to memorize these special cases. The three conditions in 5.7.1 are enough.

For T(n) = aT(n/b) + f(n) where f(n) is of the form **$\Theta(n^c \log^k(n))$**:

- If **a > b^c** → $T(n) = \Theta(n^{\log_b(a)})$
- If **a = b^c**:
  - $k > -1 \rightarrow \Theta(n^{\log_b(a)} \log^{k+1}(n))$
  - $k = -1 \rightarrow \Theta(n^{\log_b(a)} \log(\log(n)))$
  - $k < -1 \rightarrow \Theta(n^{\log_b(a)})$
- If **a < b^c**:
  - $k \geq 0 \rightarrow \Theta(n^c \log^k(n))$
  - $k < 0 \rightarrow \Theta(n^c)$

5.8 Complexities of Common Recurrence Relations

Recurrence Type	Time Complexity	Solved by
$T(n) = T(n/2) + 1$	$\Theta(\log(n))$	Master Thm (Ex. 5.18)
$T(n) = T(n-1) + 1$	$\Theta(n)$	Iterative substitution (Ex. 5.19)
$T(n) = 2T(n/2) + 1$	$\Theta(n)$	Master Thm (Ex. 5.20)
$T(n) = T(n-1) + n + 1$	$\Theta(n^2)$	Iterative substitution (Ex. 5.21)
$T(n) = 2T(n/2) + n + 1$	$\Theta(n \log(n))$	Master Thm (Ex. 5.22)

These five are worth memorizing — they cover most recursive algorithms you will meet.

Ex. 5.18 — $T(n) = T(n/2) + 1$

Master Theorem applies ($a=1$, $b=2$, $c=0$). $1 = 2^0 \rightarrow a = b^c \rightarrow \Theta(n^c \log(n)) = \mathbf{\Theta(\log(n))}$.

Ex. 5.19 — $T(n) = T(n-1) + 1 \rightarrow \Theta(n)$

Not in Master Theorem form (input is not divided) → iterative substitution.

- Step 1: $T(n) = T(n-1) + 1$
- Step 2: $T(n) = T(n-2) + 2$
- Step 3: $T(n) = T(n-3) + 3$

$$k^{\text{th}} \text{ step: } T(n) = T(n-k) + k$$

The base case is at some arbitrary constant c, so $T(n-k) = T(c)$ when $k = n-c$. With $T(c) = 1$:

$$T(n) = T(c) + (n-c) = 1 + n - c \rightarrow \mathbf{\Theta(n)}$$

Ex. 5.20 — $T(n) = 2T(n/2) + 1$

Master Theorem: $a=2$, $b=2$, $c=0$. $2 > 2^0=1 \rightarrow \Theta(n^{\log_b(a)}) = \Theta(n^{\log_2(2)}) = \mathbf{\Theta(n)}$.

Ex. 5.21 — $T(n) = T(n-1) + n + 1 \rightarrow \Theta(n^2)$

Not Master Theorem form → substitution.

- Step 1: $T(n) = T(n-1) + n + 1$
- Step 2: $T(n) = T(n-2) + (n-1) + n + 2$
- Step 3: $T(n) = T(n-3) + (n-2) + (n-1) + n + 3$

$$k^{\text{th}} \text{ step: } T(n) = T(n-k) + \Sigma_{i=0}^{k-1}(n-i) + k$$

Base case $T(c) \rightarrow k = n-c$, $T(c) = 1$:

$$T(n) = 1 + (n + (n-1) + \ldots + (c+1)) + (n-c)$$

Arithmetic sum $n((a_1+a_n)/2)$ with terms from $c+1$ to n:

$$(n-c)((c+1+n)/2) = n^2/2 + n/2 - c^2/2 - c/2$$

Drop constants and lower order terms → $\mathbf{\Theta(n^2)}$.

Ex. 5.22 — $T(n) = 2T(n/2) + n + 1$

Master Theorem: $a=2$, $b=2$, $c=1$. $2 = 2^1 \rightarrow \Theta(n^c \log(n)) = \mathbf{\Theta(n \log(n))}$.

5.9 Analysis of Recursive Algorithms

5.9.1 Matrix Partitioning (Example 5.23)

The problem

A 2-D matrix with m rows and n columns ($m \approx n$), stored as an array of arrays, sorted along **both** rows and columns: for every element, value[i][j] < value[i+1][j] and value[i][j] < value[i][j+1]. Given a target value, return whether it exists.
Question: which of three recursive algorithms is asymptotically best?

```
 1   4   7  11  15
 2   5   8  12  19
 3   6   9  16  22
10  13  14  17  24
18  21  23  26  30      (searching for 13)
```

Important

Here the input size **n is the dimension of the matrix**, **not** the total number of values, because these algorithms break the problem up by dimension size.

Four questions to build a recurrence relation from an algorithm

1. Compared to the initial input size n, what input size is passed into the recursive call?
2. How many recursive calls are made?
3. What is the time complexity of the other operations **outside** the recursive calls?
4. What is the base case?

Algorithm #1 — Quad Partition

- Split the matrix into four quadrants, compare the middle element `arr[m/2][n/2]` with the target, and eliminate the quadrant that cannot contain it. Recurse on the other three.

- Finding 13 with middle element 9: 13 > 9, so 13 must be right of or below 9 — it cannot be both left of and above 9 → the **top-left quadrant is eliminated**. Three recursive calls follow.
- **1.** Quadrant dimension ≈ n/2. **2.** 3 calls. **3.** One comparison → constant. **4.** Base case: 1×1 matrix (n = 1), constant time.

$$T(n) = 1 \text{ if } n = 1 \;\mid\; 3T(n/2) + 1 \text{ if } n > 1$$

Master Theorem (a=3, b=2, c=0): 3 > 1 → $\Theta(n^{\log_2(3)}) \approx \Theta(n^{1.585})$

Algorithm #2 — Binary Partition with Linear Search

- Also splits into four quadrants, but **linearly searches the entire middle column** to find where the target should be. If it is not in that column, eliminate the quadrant to the **upper left** and the quadrant to the **lower right** of that position → only **2** recursive calls.
- **1.** n/2. **2.** 2 calls. **3.** Linear search down the middle column → Θ(n) (since m ≈ n). **4.** 1×1 matrix, constant time.

$$T(n) = 1 \text{ if } n = 1 \;\mid\; 2T(n/2) + n \text{ if } n > 1$$

Master Theorem (a=2, b=2, c=1): $2 = 2^1$ → **Θ(n log(n))**

Algorithm #3 — Binary Partition with Binary Search

- Identical to #2, except it uses **binary search** on the middle column: check 9 first, compare with 13, and discard everything above 9 at once instead of scanning 7, 8, 9, …
- Binary search halves the search space each step → Θ(log(n)) (see chapter 15 for more).
- **1.** n/2. **2.** 2 calls. **3.** $\log_2(n)$. **4.** 1×1 matrix, constant time.

$$T(n) = 1 \text{ if } n = 1 \;\mid\; 2T(n/2) + \log_2(n) \text{ if } n > 1$$

Master Theorem cannot be used here: $f(n) = \log_2(n)$ is not of the form $\Theta(n^c)$. Use iterative substitution instead.

Solving $T(n) = 2T(n/2) + \log_2(n)$ by substitution

- Step 1: $T(n) = 2T(n/2) + \log_2(n)$
- Step 2: $T(n) = 4T(n/4) + 2\log_2(n/2) + \log_2(n)$
- Step 3: $T(n) = 8T(n/8) + 4\log_2(n/4) + 2\log_2(n/2) + \log_2(n)$

$$k^{\text{th}} \text{ step: } T(n) = 2^k T(n/2^k) + \Sigma_{i=0}^{k-1} 2^i \log_2(n/2^i)$$

Base case $T(1) \to n/2^k = 1 \to k = \log_2(n)$; $T(1) = 1$. Splitting the log with identities:

$$T(n) = n + [\log_2(n) \cdot \Sigma 2^i] - [\Sigma\, i2^i]$$

Using the geometric series and the $i2^i$ identity:

$$T(n) = n + (n\log_2(n) - \log_2(n)) - (n\log_2(n) - 2n + 2) = 3n - \log_2(n) - 2$$

Drop coefficients and lower order terms → **Θ(n)**.

★ **Shortcut (extended Master Theorem, 5.7.3, not tested):** $f(n) = \log_2(n) = \Theta(n^0 \log^1(n))$; $a=2 > b^c=1 \to \Theta(n^{\log_2(2)}) = \Theta(n)$. Same answer, far less math.

Verdict

Algorithm	Complexity	Measured runtime (1M searches, 100×100 matrix)
Quad Partition	$\Theta(n^{1.585})$	17.33 seconds
Binary Partition with Linear Search	Θ(n log(n))	10.93 seconds
Binary Partition with Binary Search	Θ(n)	6.56 seconds

- Algorithm #3 is asymptotically fastest, and the measured runtimes confirm the analysis.
- **Lesson:** it is not obvious that paying for an extra search down the middle column is worth eliminating a single recursive call — but the analysis shows it drops the complexity from **polynomial to linear**. Even an inefficient Θ(n) linear search beats making a third recursive call.

5.9.2 Tower of Hanoi

The puzzle

n disks of varying sizes stacked in sorted order on one of three rods; move them all to a destination rod under these rules:

- Only one disk can be moved at a time.
- Only the **top** disk on a rod can be moved.
- A larger disk can **never** be placed on top of a smaller one.

- **Key insight:** once the largest disk is on the destination rod, moving the remaining n−1 disks is **structurally identical to the original problem with a smaller input size** → recursion applies.
- **Base case:** n = 1 — a single disk moves trivially with no rule violated.
- **Recursive case (n > 1):**
  1. Move the n−1 smaller disks to the **auxiliary** rod (using the destination as auxiliary).
  2. Move the largest disk to the **destination** rod.
  3. Move the n−1 smaller disks from the auxiliary rod to the destination (using the original start rod as auxiliary).

Example 5.24 — time and auxiliary space

```
void tower_of_hanoi(int32_t n, int32_t src, int32_t dest, int32_t aux) {
  if (n == 1) {
    std::cout << "Moved disk 1 from rod " << src << " to rod " << dest << std::endl;
    return;
  } // if
  tower_of_hanoi(n - 1, src, aux, dest);
  std::cout << "Moved disk " << n << " from rod " << src << " to rod " << dest << std::endl;
  tower_of_hanoi(n - 1, aux, dest, src);
} // tower_of_hanoi()
```

Two recursive calls on n−1 plus constant work:

$$T(n) = \Theta(1) \text{ if } n = 1 \;\mid\; 2T(n-1) + \Theta(1) \text{ if } n > 1$$

This is exactly the recurrence from **Example 5.5** → $T(n) = 2^n - 1$ moves.

- **Time complexity: $\Theta(2^n)$**
- **Auxiliary space:** the first recursive call (line 6) is **not** tail recursive, and the recursion depth is proportional to n (decrementing from n to 1) → **$\Theta(n)$**.

Remark: the base case could equally be defined at n = 0 (do nothing and return immediately) — same solution.

Closing takeaways (chapters 4 + 5)

- Big-O analysis lets you **compare approaches before writing any code**.
- Expectations don't always mirror reality — even with the right algorithm, use tools like **perf** (which shows the percentage of total time spent in each function) to debug discrepancies.
- Big-O dictates **how runtime scales, not actual runtime**. An algorithm taking n steps is twice as fast as one taking 2n steps, even though both are Θ(n).
- Finding an efficient algorithm is **only half the battle** — you must implement it optimally, with the correct choice of data structures. A poor data structure choice can ruin an otherwise efficient algorithm.